Dominio 1 spiegato da zero: architettura agentica e orchestrazione (27%)

Questo è il dominio più pesante dell'esame: più di un quarto delle domande. Il modo più semplice per capirlo è partire da una domanda: **cos'è un agente?**

> **Prima di iniziare.** Questa pagina è scritta anche per chi non programma. Quando incontri un concetto tecnico "di base" trovi il simbolo → **Pre-corso** con un link alla spiegazione completa. Per questo dominio conviene aver letto almeno:
>
> - Funzioni, parametri e valori di ritorno (cos'è uno strumento, visto dal lato del codice);
> - Cicli e condizioni (il loop agentico è un ciclo);
> - Deterministico e probabilistico (l'idea più importante di tutto l'esame);
> - JSON (il formato in cui viaggiano richieste e risultati).
>
> Per i termini specifici di Claude tieni aperto il Vocabolario.

1. Prima di tutto: cos'è un agente?

Un modello linguistico, come Claude, di suo sa solo rispondere con testo. Sa scrivere, ragionare, riassumere. Non può fare nulla nel mondo: non può aprire un file, non può consultare un database, non può rimborsare un cliente.

→ Pre-corso: Modello linguistico (LLM)

Un **agente** è Claude messo in un ciclo, con la possibilità di usare degli **strumenti** (in inglese *tools*). Uno strumento è una funzione che il programmatore mette a disposizione: per esempio "cerca un ordine per numero", "invia un rimborso", "leggi un file".

Una metafora utile: pensa a un assistente in ufficio. Non ha le mani sul sistema informativo. Quando gli serve un dato, chiede alla segreteria di tirarlo fuori, la segreteria glielo porta, lui continua. L'agente funziona così: chiede di usare uno strumento, il programma lo esegue, il risultato torna a Claude, e Claude decide il passo successivo.

Chi fa cosa: Claude decide, il programma esegue

È il punto che confonde di più chi non programma. **Claude non esegue mai direttamente uno strumento.** Le parti in gioco sono tre:

Chi	Ruolo	Nella metafora dell'ufficio
Claude	Ragiona e decide quale strumento serve, con quali dati	L'assistente che sa cosa chiedere
Il programma (l'applicazione scritta dallo sviluppatore)	Esegue davvero lo strumento e riporta il risultato	La segreteria che apre l'archivio

Chi	Ruolo	Nella metafora dell'ufficio
Il sistema esterno (database, gestionale, file)	Contiene i dati o compie l'azione	L'archivio

Questa separazione è preziosa: siccome è il programma a eseguire, il programma può anche **rifiutarsi** di eseguire. È la base di tutto il discorso sui controlli (sezioni 4 e 5).

Un esempio passo per passo

Un cliente scrive: "*Dov'è il mio ordine 67890?*". Ecco cosa succede davvero.

1. Il programma invia a Claude il messaggio del cliente e l'elenco degli strumenti disponibili (con nome e descrizione).
2. Claude risponde: "*Voglio usare* `lookup_order` *con* `order_id = 67890`". Non ha ancora risposto al cliente: ha solo chiesto uno strumento.
3. Il programma esegue `lookup_order` nel gestionale e ottiene: "*spedito il 3 marzo, corriere X*".
4. Il programma rimanda a Claude questo risultato.
5. Claude ora ha il dato e risponde al cliente: "*Il tuo ordine è stato spedito il 3 marzo con il corriere X*". Questa volta dichiara di aver finito.

I passi 2–4 possono ripetersi più volte (prima cerca il cliente, poi l'ordine, poi la spedizione). Questa ripetizione è il **loop agentico**.

Due parole che userai continuamente:

- **Messaggio**: ciò che scambiate. Ci sono messaggi dell'utente e messaggi di Claude. Anche i risultati degli strumenti tornano a Claude dentro un messaggio "utente".
- **Turno**: un passaggio completo, dalla richiesta alla risposta finale.

---

2. Il loop agentico: il cuore del dominio

Il **loop** è il ciclo che si ripete: Claude risponde, se ha chiesto uno strumento il programma lo esegue e rimanda il risultato, Claude risponde di nuovo, e così via finché non ha finito.

→ Pre-corso: Cicli e condizioni, dove trovi anche il loop scritto in codice.

La domanda decisiva è: **come fa il programma a sapere se Claude ha finito?**

La risposta giusta è il campo `stop_reason` (letteralmente "motivo di fermata"). Ogni risposta di Claude dichiara il motivo per cui si è fermato. È un'etichetta tecnica, pensata per essere letta da un programma, non da una persona. I valori da conoscere:

`stop_reason`	Significato semplice	Cosa fa il programma
`tool_use`	"Voglio usare uno strumento"	Esegue lo strumento e continua il loop
`end_turn`	"Ho finito"	Esce dal loop

`stop_reason`	**Significato semplice**	**Cosa fa il programma**
`max_tokens`	"Mi sono interrotto perché ho finito lo spazio"	La risposta è tagliata: va gestita
`pause_turn`	"Un processo lato server si è fermato a metà"	Rimanda la stessa risposta per far continuare
`refusal`	"Mi rifiuto di rispondere"	Va gestito esplicitamente

→ Pre-corso: Token e finestra di contesto, per capire cosa significa "finito lo spazio" (`max_tokens`).

Trappola numero uno dell'esame: il loop non deve uscire quando Claude scrive frasi tipo "ho finito" o "posso aiutarti altro?". Il testo può essere ingannevole: a volte una risposta contiene del testo *e* una richiesta di strumento. L'unico criterio affidabile è `stop_reason`.

Perché il testo inganna? Perché è scritto per le persone. Claude può scrivere "*Perfetto, controllo subito l'ordine*" e, nella stessa risposta, chiedere lo strumento. Se il programma vedesse quella frase e chiudesse il loop, il cliente non riceverebbe mai la risposta vera. Il campo `stop_reason`, invece, è un'etichetta strutturata con pochi valori possibili: non c'è nulla da interpretare.

Etichette e ricevute: `tool_use_id`

Ogni richiesta di strumento ha un'etichetta (`tool_use_id`). Quando restituisci il risultato, devi riportare esattamente la stessa etichetta. È come una ricevuta del guardaroba: se il numero non corrisponde, il sistema non sa a quale richiesta si riferisce il risultato.

Se Claude chiede tre strumenti insieme, le etichette permettono di abbinare ciascun risultato alla sua richiesta, anche se arrivano in ordine diverso.

Quando lo strumento fallisce: `is_error`

Se lo strumento fallisce, si segnala con `is_error: true`, così Claude sa che è un errore e non un dato vero. Un risultato del tipo "*servizio non disponibile*" senza questa etichetta potrebbe essere scambiato per un'informazione. Con l'etichetta, Claude può riprovare, cambiare strada o avvisare l'utente. Il dominio 2 approfondisce come scrivere errori utili.

Il limite di sicurezza

Puoi mettere un numero massimo di giri, per evitare che il loop giri all'infinito (per esempio se due strumenti si rimandano la palla). Ma è una protezione di ripiego, non la condizione normale di uscita. Chi usa solo un contatore sbaglia l'esame: se il limite è 5 e il compito ne richiede 6, il lavoro viene troncato; se il compito ne richiede 2, il contatore non serve a capire che è finito.

---

3. Più agenti: il modello hub-and-spoke

Quando un compito è troppo grande per un solo agente, si usano più agenti. Il modello dell'esame è **hub-and-spoke**, cioè a raggiera:

- Un **coordinatore** (l'hub, il mozzo di una ruota) riceve la richiesta, la scompone in parti, assegna ciascuna parte a un **subagente** (i raggi, gli *spoke*), raccoglie i risultati e produce la risposta finale.
- I **subagenti non parlano tra loro**. Tutto passa dal coordinatore.

→ Pre-corso: Orchestrazione

Un'analogia: un capo progetto che assegna pezzi di lavoro a tre collaboratori. I collaboratori non si scrivono tra loro direttamente: ogni informazione passa dal capo. Questo rende il sistema più controllabile: c'è un solo punto in cui vedere tutto, verificare e correggere.

Perché usare più agenti?

Non è una questione di "più è meglio". I vantaggi reali sono:

- **Specializzazione**: ogni subagente ha poche istruzioni e pochi strumenti, adatti al suo compito, e quindi sbaglia meno (il dominio 2 spiega perché troppi strumenti confondono).
- **Contesto pulito**: un subagente può leggere 50 documenti e restituire un riassunto di dieci righe. Il coordinatore non si riempie di materiale inutile (vedi il dominio 5).
- **Parallelismo**: compiti indipendenti possono procedere insieme, riducendo i tempi.

Il prezzo è un sistema più complesso: più chiamate, più costi, più punti in cui l'informazione può perdersi. Per un compito semplice, un solo agente è la scelta giusta.

Il coordinatore valuta il risultato

Il coordinatore ha anche un dovere: **valutare il risultato**. Se la somma dei pezzi non copre tutto il problema, deve rilanciare il lavoro. Esempio: se devi ricercare "industrie creative" ma i subagenti hanno coperto solo le arti visive, la scomposizione era troppo stretta. Mancano musica, editoria, cinema, design. La colpa non è dei subagenti, che hanno fatto bene ciò che era stato chiesto: è del coordinatore, che ha diviso male.

Il punto più importante: il contesto non si eredita

Questa è la trappola più frequente. Un subagente **parte da zero**. Non ha letto la conversazione del coordinatore. Non sa chi è il cliente, cosa ha già detto, quale ordine è coinvolto.

Quindi, quando deleghi un compito, devi scrivere nel prompt del subagente **tutto** ciò che serve: ID cliente, numero d'ordine, importo, richiesta precisa. Un prompt come "analizza il problema del cliente" è inutile, perché il subagente non sa quale cliente.

Un confronto concreto:

Delega sbagliata	Delega corretta
"Verifica se il cliente ha diritto al rimborso."	"Verifica se il cliente con ID 12345 ha diritto al rimborso dell'ordine 67890 (importo 149,99 €, consegnato il 2 marzo). Il cliente dichiara che il prodotto è arrivato danneggiato. Applica la policy resi a 30 giorni. Restituisci: idoneo sì/no, motivo, eventuali documenti mancanti."

Pensa a quando passi una pratica a un collega appena arrivato: non puoi dirgli "occupati di quella cosa". Gli devi dare il fascicolo.

Nel materiale, lo strumento che il coordinatore usa per lanciare un subagente si chiama `Agent`. Nei materiali più vecchi e nella guida d'esame compare come `Task`: è lo stesso meccanismo, cambia solo il nome.

Parallelo o sequenziale?

→ Pre-corso: Parallelo e sequenziale

Se vuoi che tre subagenti lavorino **contemporaneamente**, il coordinatore deve inviare le tre chiamate `Agent` **nella stessa risposta**. Se le manda in tre turni diversi, lavorano uno dopo l'altro, e il sistema è più lento.

Il parallelo ha senso solo se i compiti sono **indipendenti**. Se il secondo compito ha bisogno del risultato del primo (prima trova i documenti, poi analizzali), devono per forza essere in sequenza.

Fork o nuovo subagente?

- Un **fork** è una copia del lavoro che eredita tutta la conversazione e condivide la cache (la memoria veloce di cui parleremo nel dominio 4). Serve per esplorare due strade diverse partendo dalla stessa base.
- Un **nuovo subagente** parte isolato. Serve per un compito circoscritto.

Una metafora: il fork è come fotocopiare il fascicolo completo per due colleghi che proveranno due soluzioni diverse; il nuovo subagente è come dare a un consulente esterno solo la scheda che gli serve.

Attendibilità

Quando un agente passa informazioni a un altro, conviene separare il **contenuto** dai **metadati**: cosa è stato affermato, da quale fonte, con quale data. Così chi riceve può valutare quanto fidarsi. Un "il fatturato è 2,3 miliardi" senza fonte non vale quanto "il fatturato è 2,28 miliardi secondo il bilancio certificato del 2024". Il dominio 5 riprende questo tema (la provenienza).

---

4. Regole vere vs regole scritte: il concetto più verificato

Qui c'è una distinzione fondamentale.

→ Pre-corso: Deterministico e probabilistico. Se leggi una sola sezione del pre-corso, leggi questa.

- **Applicazione basata sul prompt**: scrivi la regola nelle istruzioni. Claude la segue *di solito*. È **probabilistica**: "di solito" non basta quando un errore costa denaro.
- **Applicazione programmatica**: il programma impedisce fisicamente l'azione se la condizione non è soddisfatta. È **deterministica**: non può sbagliare.

Un esempio concreto. Regola: "non spedire un ordine se il pagamento non è confermato". Se la scrivi nel prompt, in qualche test l'agente spedisce prima di verificare. Se metti un controllo nel programma che blocca la spedizione finché il pagamento non è confermato, l'errore è impossibile.

Un'analogia: un cartello "vietato l'ingresso ai non autorizzati" è una regola scritta; quasi tutti la rispettano, ma qualcuno entra lo stesso. Un tornello con il badge è una regola applicata: chi non ha il badge non passa, punto.

Non significa che il prompt sia inutile. Il prompt serve per **guidare il giudizio**: il tono con il cliente, come spiegare una policy, quando fare una domanda in più. Il codice serve per ciò che **non può mai fallire**.

Esempio di regola	Dove va
"Sii cortese e spiega i motivi del rifiuto"	Prompt
"Verifica l'identità prima di qualunque rimborso"	Codice (hook o controllo nello strumento)
"Rimborsi sopra i 500 € richiedono un operatore umano"	Codice
"Se il cliente è confuso, fai un esempio"	Prompt

Regola d'oro: per operazioni finanziarie o di sicurezza, usa il controllo programmatico, non il prompt. Nelle domande d'esame, parole come "sempre", "mai", "garantire", "conformità" indicano quasi sempre la risposta deterministica.

---

5. Gli hook: il meccanismo del controllo programmatico

Un **hook** (letteralmente "gancio") è un pezzo di codice che il programma esegue automaticamente in momenti precisi del ciclo. Pensalo come un **buttafuori** all'ingresso di una sala: controlla ogni richiesta prima che entri.

L'hook non dipende dalla volontà di Claude: scatta sempre, a ogni richiesta di strumento. Per questo è il modo concreto di trasformare una regola scritta in una regola applicata.

Gli hook da conoscere:

- `PreToolUse`: si attiva **prima** che uno strumento venga eseguito. Può bloccarlo. Per bloccare, il comando esce con codice `2`, oppure restituisce la decisione `"deny"` (negare). Altri valori possibili: `"allow"` (consenti) e `"ask"` (chiedi conferma).
- `PostToolUse`: si attiva **dopo** lo strumento, prima che Claude legga il risultato. Utile per ripulire i dati o normalizzare un formato, per esempio togliendo dati sensibili.
- `PostToolUseFailure`: gestisce i casi in cui lo strumento è fallito.
- `Stop` e `SubagentStop`: segnano la fine del turno o del subagente.

→ Pre-corso: stdin, stdout, stderr ed exit code, per capire cosa significa "esce con codice 2". In breve: ogni programma, quando termina, restituisce un numero; `0` vuol dire "tutto bene", altri numeri segnalano altro. Per gli hook di Claude Code, `2` significa "blocca".

→ Pre-corso: Dati personali e sensibili, per capire perché `PostToolUse` è utile a togliere numeri di carta o codici fiscali prima che Claude li legga.

Ci sono vari tipi di hook (comando, chiamata web, strumento MCP, prompt, agente). Non serve memorizzarli tutti per l'esame.

Esempio tipico d'esame

Vuoi impedire un rimborso senza aver prima verificato l'identità del cliente. Metti un hook `PreToolUse` sullo strumento `process_refund`, che controlla che lo strumento `get_customer` abbia già restituito un ID verificato. Se non è verificato, il rimborso viene bloccato.

Cosa succede, in sequenza:

1. Claude, magari convinto dal cliente, chiede `process_refund`.
2. Prima che lo strumento parta, scatta l'hook `PreToolUse`.
3. L'hook controlla: c'è un ID cliente verificato in questa sessione? No.
4. L'hook risponde `"deny"`, con un messaggio tipo "*Verifica prima l'identità con get_customer*".
5. Claude riceve il rifiuto, chiama `get_customer` e solo dopo riprova il rimborso.

Nessuna formulazione del prompt, per quanto insistente, offre la stessa garanzia.

Dove si configurano

In `~/.claude/settings.json` (personale), in `.claude/settings.json` (condiviso nel repository, quindi con il team), in `.claude/settings.local.json` (locale, escluso da Git) e nelle policy gestite dall'amministratore.

→ Pre-corso: File system, percorsi e glob (cosa significa `~`) e Repository e Git (cosa significa "condiviso nel repository" ed "escluso da Git").

Il dominio 3 e l'approfondimento sugli hook entrano nei dettagli della configurazione.

---

6. Come scomporre un problema

Ci sono tre modi di spezzare un lavoro:

1. **Concatenazione di prompt**: una sequenza fissa di passi, quando sai già quali sono. Come una catena di montaggio. Esempio: estrai i dati dalla fattura → verifica i totali → traduci il riepilogo in inglese. I passi sono sempre gli stessi.
2. **Scomposizione adattiva**: i passi successivi dipendono da ciò che emerge. Come un'indagine: ogni scoperta apre nuove domande. Esempio: "perché le vendite sono calate a marzo?". Non sai in anticipo se la causa sarà il prezzo, un concorrente o un problema di consegne: ogni risultato decide la prossima verifica.
3. **Revisione in più passaggi**: per una PR (una modifica al codice) grande, un passaggio analizza ogni file da solo, un secondo controlla come i file si parlano tra loro. Un passaggio unico perde attenzione sui file lontani e non vede i problemi tra file.

→ Pre-corso: GitHub (cos'è una pull request) e Test, lint e code review.

Come scegliere: se riesci a scrivere i passi su un foglio **prima** di iniziare, usa la concatenazione (è più prevedibile e più facile da controllare). Se i passi dipendono dalle scoperte, serve la scomposizione adattiva.

---

7. Gestire le sessioni

Una **sessione** è una conversazione con Claude Code che ha memoria del lavoro fatto. Puoi riprenderla o cambiarla:

- `--resume`: riprende la sessione con tutto il suo contesto. Ha senso se i risultati precedenti sono ancora validi.
- **Nuova sessione con riepilogo**: se i file sono cambiati o il contesto si è allontanato, meglio ricominciare da un riassunto strutturato.

- `--fork-session`: crea un ramo con un nuovo ID per provare due strategie diverse partendo dalla stessa base.

→ Pre-corso: Argomenti e flag. Le parole che iniziano con `--` sono opzioni che si aggiungono al comando `claude` nel terminale.

Attenzione: se riprendi una sessione dopo aver modificato dei file, devi dirlo esplicitamente a Claude. Non se ne accorge da solo, e potrebbe basarsi su informazioni vecchie. È come tornare da un collega dopo una settimana: lui ricorda la versione del documento che avevate visto insieme, non quella che hai modificato nel frattempo.

Come scegliere:

Situazione	Scelta
Riprendi domani lo stesso lavoro, nulla è cambiato	`--resume`
Il codice è cambiato molto, o la conversazione era piena di tentativi falliti	Nuova sessione con un riepilogo
Vuoi provare due soluzioni alternative dallo stesso punto	`--fork-session`

L'approfondimento sulle sessioni (sezione *Approfondimenti*) entra nei dettagli.

---

8. Le cinque cose da ricordare

1. Il loop finisce su `stop_reason`, non su frasi o contatori.
2. Il contesto dei subagenti va scritto esplicitamente: non lo ereditano.
3. Subagenti in parallelo = più chiamate `Agent` nella stessa risposta.
4. Per soldi e sicurezza, usa hook programmatici, non solo istruzioni.
5. I subagenti non parlano tra loro: tutto passa dal coordinatore.

Per capire meglio: le domande d'esame di questo dominio

Le domande del file tipicamente ti mettono davanti a un problema reale (un agente che si ferma troppo presto, un subagente che ignora le preferenze, una regola che viene saltata) e ti chiedono la correzione più solida. La risposta quasi sempre è quella **deterministica** (un controllo nel codice, un criterio di uscita corretto, un contesto passato esplicitamente) e non quella che si affida a "dire meglio" a Claude nel prompt.

Tre ragionamenti tipici

"L'agente a volte risponde al cliente prima di aver controllato l'ordine." Il sintomo è un loop che esce troppo presto. Cerca la risposta che usa `stop_reason` invece di frasi nel testo o di un contatore.

"Il subagente ignora le preferenze che il cliente aveva espresso al coordinatore." Il subagente non le ha mai viste. La risposta giusta è passarle esplicitamente nel prompt di delega, non "ricordare al subagente di leggere la conversazione".

"Nonostante le istruzioni, ogni tanto parte un rimborso senza verifica." Le istruzioni sono probabilistiche. La risposta giusta è un hook `PreToolUse` (o un controllo nel codice), non un prompt più severo o scritto in

maiuscolo.